GRAVID-AMEN: Building, Deploying and Handing Over a Maternal Health Application

Ella Thorburn
University of Cape Town
Department of Computer Science
South Africa
threll002@myuct.ac.za

Abstract

This paper reports on the development and handover of GRAVID-AMEN, an offline-first maternal nutrition application for rural Limpopo, South Africa. Requirements were formalised from formative research conducted by the AMEN team at the University of Limpopo and an inherited product requirements document. Maintainability requirements were elicited from literature, two UCT HCI Lab projects, and retrospective interviews. A field build was evaluated with 51 participants on 34 devices at three clinics. Field observations showed that copying a familiar layout did not make the affordances of its components evident. A post-fieldwork build added content management, offline synchronisation and dependency monitoring. The application was then handed over to a developer employed by the AMEN team. The findings indicate that documented, runnable code is insufficient for continuity. Continuity depends on artefact access remaining with the project, recorded reasoning reaching a receiving developer, and maintenance notices reaching someone responsible and able to act.

Keywords

Software Maintainability, Software Handover, Offline-First mHealth, Content Management, Maternal Health, Low-Resource Settings, Student Software Projects

1 Introduction

This project aims to deliver a pilot-ready version of GRAVID-AMEN: a mobile application providing culturally relevant nutrition education to pregnant women, their families, and community health workers (CHWs) in rural Limpopo. The development was proposed by researchers from the AMEN project, a maternal nutrition initiative at the University of Limpopo (UL), which identified a need for this mobile health (mHealth) intervention through its research [18] in rural communities outside Polokwane. An essential requirement was that the application be maintainable and scalable so that it could support future studies and a planned government handover.

The AMEN project originally partnered with a team from a university in Nigeria to build the application. That collaboration ended, creating a need for a new development partner to be found. Associate Professor Melissa Densmore from The University of Cape Town (UCT) Human Computer Interaction (HCI) Lab was approached, and offered the work as an honours project under her supervision. The UCT team was to develop the application and design the interaction protocol for assessing the application in the field, and the AMEN team was to supply the culturally relevant nutrition content and Sepedi translations, as well as organise logistics for the field work.

The Product Requirements Document (PRD) the AMEN team provided to develop the application against was made by the initial Nigerian development partners. It outlines an adaptation of GRAVID, a maternal nutrition application built for the Nigerian context [2]. The adaptation contextualises some features and content to Limpopo. The PRD, as well as formative meetings with the AMEN team, yielded the following aims:

Aim 1 (design and development). Develop GRAVID-AMEN against the functional and non-functional requirements of the PRD, first to a version fit for use in the field work and then to one ready for continued development and deployment.

Aim 2 (maintainability). Build GRAVID-AMEN to be maintainable and in a state a receiving developer continuing the work for the AMEN project could set up, understand and iterate on.

The PRD specified the functional requirements but not what maintainability would require, so that was elicited from two earlier UCT Human Computer Interaction (HCI) Lab projects and from the literature. The application was developed through three phases: a field build used during the field work in Limpopo, a post-fieldwork build prepared for handover, and a pilot-ready build written by the receiving developer at the UL and piloted in August. This is one of two papers from a two-person project. It covers the development process of the application, including technical details from the field deployment, and the handover. Sarah-Jane Marais's paper covers the interaction plan for the field work and its results. While this is primarily a software engineering paper, the retrospective analysis, the interviews with the people behind those two earlier projects, and the handover of this application each produced insights, which are reported here for future projects to learn from.

2 Background and Related Work

2.1 mHealth Interventions in Low-Resource Settings

Mobile health (mHealth) interventions are widely deployed in response to maternal and child mortality still being high in Sub-Saharan Africa compared to developed countries [1, 26]. Many fail, largely because they do not account for the infrastructural and cultural realities of the people they serve [1] and because the intervention is difficult to sustain after it launches [19]. Till et al. [26] reviewed 141 articles across low- and middle-income countries and found most interventions rely on SMS or USSD. They argue for applications that deliver audio and video instead, because text excludes users who cannot read it and content held on the device avoids the recurring cost of SMS and USSD. Adusei-Mensah et al. [1] reviewed 63 studies across Sub-Saharan Africa and found the

larger problem to be that interventions do not fit the cultural or the infrastructural realities of the people they serve. South Africa's national maternal health service, MomConnect, reaches women through SMS, USSD and WhatsApp [4], but that messaging base has grown expensive as it scaled [20] and cannot deliver the video and audio this audience asks for. Dsane-Nsor, Félix and Farao each find that mothers trust and act on health information that reaches them through recognisable community voices and through narrative far more than on generic instruction, even from professionals [9, 10, 12]; Félix et al. add that parents weigh advice against their own values and dismiss anything generic or judgemental. Low literacy is difficulty reading text in any medium. It is accommodated by audio, images and less text. Low digital literacy is unfamiliarity with interface conventions and is accommodated by using fewer controls and larger targets in user interfaces (UI) [17].

2.2 The Infrastructural and Cultural Reality of the Target Users

Muthelo et al. [18] is the study closest to this project and the one the requirements cite. Eighteen mothers, community health workers and fathers were interviewed across three rural villages outside Polokwane, one of them Dikgale, which is zoned rural and low income, is ninety-five percent Sepedi speaking [5], and is one of this project's field sites. Data cost, poor network coverage and device storage were the barriers to information access her participants raised most often, and five of the eighteen reported no access to a smartphone of their own. They asked for recorded voice and video over text: video travels quickly through the platforms they already use, and a mother who reads with difficulty can listen while doing something else, provided it is in her own language. They also raised MomConnect's limits, that it sends messages out from the centre but is not designed to let a user interact with it [4]. Community health workers are a person's first point of contact with primary health care and are rooted in the communities they serve [18], visiting mothers at home through pregnancy and after birth to give education and support [27]. Coleman et al. [5] ran generative co-design workshops across four South African sites, one of them Dikgale, in which participants ranked maternal and child health challenges and then brainstormed solutions. Parents at all four sites said the information available to them is too text-heavy, too general, not in their own language and not matched to their own child's needs. The study also found automated translation to be unreliable for South African languages, and argues that first-language speakers must verify all translations and be involved in creating content from the start. Till et al. [25] report the methodological lessons from those same workshops. Participants did not take up technologies they did not already know: internet-in-a-box, community wireless networks and chatbots were foreign to these communities, and participants fell back on mobile data and WiFi, the concepts they already knew. At the Cape Town site they record fathers who wanted to be involved but found that maternal and child health information and programmes were not meant for them.

2.3 Maintainability in University-Led Software Projects

Two multi-year projects in the UCT HCI Lab have faced maintainability challenges. Milk Matters (MM) was a donor-facing application for a Cape Town human milk bank, and the Bhabhisana Baby Project (BBP) was an early childhood development platform for a Western Cape organisation. Both were built for South African non-profit partners by successive groups of students, and neither was running at the time of writing. Fehr et al. [11] address the handover of research software written by students and short-term staff and set a minimum standard for codebase state at handover: the code is available, its ownership is clear, its execution environment is documented, a working example exists and minimal documentation travels with it. They also treat the overlap between one developer leaving and the next arriving as part of the work. Jansen and Bosch [16] argue that a software architecture is properly understood as the set of design decisions that produced it. Representing it only as components and connectors leaves those decisions implicit, so the knowledge behind them vaporises as the system ages and its authors move on. Talbot and Densmore [24] set out the challenges of university-led mobile application development for non-profit organisations. They record that access to the accounts the software runs on, the reasoning behind design decisions and the continuity of development itself tend to be lost when these projects change hands, and make three recommendations: administrative access held by someone who remains at the university, signing keys and project information kept where a supervisor can reach them, and permission to retain student contact details after graduation. Talbot [23] documents the case they came from, across seven years of student projects (Appendix A.7). Sibeko [22] found the previous iteration's proposal and papers were not enough to understand the system and argued that a real handover must pass on the previous team's context, domain knowledge, design decisions and lessons. Banganayi [3] met the same difficulty, chose a rebuild language for how easily a later team could continue it, and proposed that developers record cultural requirements alongside functional and non-functional ones.

3 Requirements Analysis

3.1 Interpretation of the PRD

The PRD was only partially adapted from its Nigerian origins and contained contradictions. For example, it named the application's second language as Swahili for a Sepedi-speaking region. These contradictions meant the requirements had to be audited against the infrastructural and cultural realities of the end users. This was done by (1) analysing the formative research and linking it to formal user conditions (UCs), (2) taking out content requirements that the PRD presented as functional requirements, and (3) transforming the requirements into an actionable and verifiable format. The requirements' original form is in Table 5 and Table 6, and Table 7 and Table 8 trace each one to where it came from in the PRD.

3.2 User Conditions Linked to the Formative Research

Table 1: User conditions, gathered from formative research and community interviews.

User Conditions
UC1. Data is a significant expense for users [18].
UC2. Network coverage is intermittent [8, 18].
UC3. Device storage is limited [18].
UC4. Phones are often shared between household members, and not always personally owned [18].
UC5. Video and audio content is preferred over text [18].
UC6. Unfamiliar interaction patterns are avoided; simple, already-familiar ones are adopted readily [18, 25].
UC7. Users want personalised, individually-relevant content rather than generic guidance [5].
UC8. Users want two-way, interactive communication rather than one-way broadcast messages [18].
UC9. Fathers and other family members want involvement in MCH, but existing content and programmes are designed only for mothers [5].
UC10. Sepedi is the predominant home language in this community [5, 18].

3.3 Functional Requirements

Table 2: Functional requirements, reworked from the PRD.

Functional Requirements
Onboarding
FR1. A user can register with a phone number, verified by SMS one-time passcode. *Rejected*
FR2. A user can select a display language at registration and change it any time in settings.
FR3. A user can select a role at sign-up (pregnant woman, partner/family member, elder, CHW, nurse).
FR4. The application captures a pregnancy stage or due date on the profile.
FR5. A user can create and manage more than one linked profile (e.g. a CHW managing client profiles).
Content Management
FR6. The application supports content items in any combination of text, audio, image and video.
FR7. The application supports tagging content with one or more categories (topic, theme, stage, belief, coping strategy).
FR8. The application supports a reusable two-sided card type, a statement on the front and a reveal action on the back.
FR9. A user can record or upload their own testimonial (text, audio or video).
Content Delivery
FR10. The application presents interface text and content in the selected language.
FR11. The application customises feature and content display by profile role.
FR12. A user can select a category to view the content tagged with it.
FR13. The application supports caption/subtitle tracks on video and audio.
Settings
FR14. A user can adjust the font size for text content.
FR15. A user can adjust the playback speed of audio and video.
FR16. A user can select a low-literacy mode that defaults to audio and image over on-screen text.
FR17. A user can enable or disable each reminder category independently.
Reminders
FR18. The application schedules reminders on a fixed interval (a daily learning nudge), a profile date or stage (ANC visit prompts by pregnancy stage), or a user-logged event (supplement intake).
FR19. A user can log meals or food intake against their profile.
Messaging
FR20. A user can message a professional account (e.g. a nurse) and receive a reply.
FR21. A professional account can view and reply to messages from users.
Offline
FR22. The application keeps bundled or downloaded content viewable offline, and needs no connection to display the entry screen, switch language, or switch profile.
FR23. A user can download content for offline use.
FR24. The application synchronises offline data to the backend once connectivity returns.
Analytics
FR25. The application collects usage analytics.

3.4 Non-functional Requirements

Table 3: Non-functional requirements, reworked from the PRD.

Non-Functional Requirements
Performance
NFR1. The application encodes and delivers video and audio within a bitrate/file-size threshold suitable for 2G/3G networks.
NFR2. Each content bundle stays under 2MB. *Rejected*
NFR3. The installed package stays under 50MB.
NFR4. The application runs on Android devices under 2GB RAM.
NFR5. The application displays its entry screen within 3 seconds of opening from the home screen.
Usability
NFR6. The application shows a visible state change on touch.
NFR7. Interactive elements have a minimum touch target of 48x48dp.
Security
NFR8. Data stored locally on the device is encrypted.
NFR9. The application complies with GDPR and POPIA data protection requirements.
NFR10. Usage analytics under FR25 are anonymised, with no data stored or transmitted in an identifiable form.
Scalability
NFR11. The application supports at least 5,000 concurrent users in the pilot phase.

3.5 Maintainability Requirements

The maintainability requirements were elicited by firstly reading the codebases and student papers of the two previous multi-year HCI Lab projects before development began, alongside the literature reviewed in Section 2.3. Then they were gathered through semi-structured remote interviews with members of those projects (a 2021 MM developer, a 2023 BBP developer and Densmore) in August, after the field study. Professor Muthelo was also interviewed to provide a non-technical partner's perspective. MR1 rests on two cases from the previous projects. The 2021 MM developer inherited a working system that was not in a language they had experience with, so they learnt the syntax to be able to iterate on the system. In contrast, the 2023 BBP developer inherited a system whose core features did not work, and decided to rebuild the system in Java. He reported in his interview that had it been working, his group would have learned the language of the original system instead of rebuilding. These cases provide evidence that what the language costs a successor is a function of whether what they inherit runs. MR5 comes from three accounts of where access ends up. The 2021 MM developer still held live credentials to her project's backend two years after leaving. The 2023 BBP group put every password the project used into one document and emailed it to Densmore at the end of the project, which that developer describes as the only option open to them, since there is nowhere in the institution to put such a document and student accounts are deleted within months of graduation. The AMEN project lead, who has never left the project, reported not knowing where its code is stored and not having access to it, and the same of an earlier application in the same research network. MR7 comes from a deprecation and security notice that Firebase sent about the MM backend after the 2021 project ended. It reached a graduate who understood what it meant and who took no action, assuming the supervisor received it too, and Densmore's account of acting on such a notice is that she would then have needed somebody able to check the software and bring it back up. The project had expired by the time the next developer opened it.

Table 4: Maintainability requirements, gathered from codebase review and stakeholder interviews.

Maintainability Requirements
MR1. The code is written in a popular language that a successor is likely to know (Interviews: 2023 BBP Developer).
MR2. Content can be changed by a non-technical partner without a developer present [23].
MR3. There is a remotely stored repository under version control [23].
MR4. The reasoning behind the code is documented and not only the behaviour [16, 22].
MR5. Administrative access is held by someone who remains at the institution [24].
MR6. An overlap occurs between the outgoing and the incoming developer [11].
MR7. Deprecation notices reach an individual whose job it is to act on it and has the capacity to do so (Interviews: Densmore, 2021 MM Developer).

3.6 Rejected Requirements

Registration by phone number, verified with an SMS one-time password (OTP) (FR1) requires a connection at first launch. Implementing this would contradict with the required offline-first operation (FR22), which is essential to addressing that data is a significant expense for users (UC1) and network coverage is intermittent (UC2). An SMS OTP may also be an unfamiliar interaction pattern to users that risks deterring them from using the application (UC6). These reasons led to the rejection of FR1. The design implication is that the application should sign a user in anonymously, and no account is required. NFR2 requires that each content bundle stays under 2MB. Meeting this requirement would heavily restrict users from downloading the preferred form of content (UC5), as audio and video files of a usable quality are seldom less than 2MB. NFR2 is therefore rejected, allowing users to download any content for offline use that they choose (FR23).

4 The Field Build

4.1 Technology Selection

A Xiaomi Redmi 9A running Android 10 with 2.71GB RAM was used to test the application throughout development, as it was an entry-level device with specifications comparable to the devices described by Muthelo et al. [18]. React Native was chosen for implementation as it is one of the most used cross-platform native frameworks [15], and it is the language the framework Expo is built on. Expo is a maintained and well-documented development framework that allows usage of tools and libraries that rapidly speed up cross-platform development. Its libraries were used for video and audio playback, navigation, file access and media storage. It also provides usage of Expo Go, a companion application installed on a mobile phone that loads a project running on the developer's machine over the local network. This allows interaction with the app as though it has been installed on the device. Changes to the code could therefore be checked on the Redmi instantly, saving valuable time by not needing to build a new APK and install it manually for every change. Flutter was considered as an alternative but not chosen as the benefits of the Expo framework were well beyond what Flutter could provide.

4.2 Scoping the Field Build

The field build deliberately did not attempt to meet all the outlined requirements. The study sites were reported to have intermittent connectivity (UC2), so to minimise the risk of the field work being disrupted, the field build was constrained to operating fully offline. This ruled out implementing: downloading content for offline use (FR23), synchronising offline data back to the backend (FR24), recording or uploading a testimonial (FR9), messaging a professional account (FR20, FR21), and collecting usage analytics (FR25). Reminders (FR17, FR18) were left out as they only take effect across days or weeks, so nothing a single supervised session could observe could effectively show their functionality. In the first fieldwork activity, participants interacted with the same maternal nutrition content presented through the application, YouTube, and TikTok, and then ranked the three platforms according to their preferred way of receiving this content. The field build therefore needed to implement features which distinguished it from the other platforms. Supporting content items in any combination of text, audio, image and video (FR6), tagging and sorting content with categories, and customising feature and content display by profile role, language, and pregnancy stage (FR11, UC7), were prioritised. Implementing this functionality required building onboarding to collect the customising information. This approach contrasts YouTube and TikTok's, which customises user content algorithmically based on users' content interactions. Not utilising user inputted information limits their ability to deliver specific and meaningful content. The field build was constrained by time and limited to one developer, which forced the remaining requirements out of scope to ensure delivery of the application by the trip to Limpopo. This excluded two-sided cards (FR8), subtitles (FR13), adjusting font size and playback speed (FR14, FR15), low-literacy mode (FR16), and meal logging (FR19).

4.3 Implementing Content Delivery

To implement the prioritised features, an assets folder containing the content was compiled into the application for the field build. A data model (Appendix A.15) was designed that attached attributes to the content items, such as language, media type, tags, and which role and pregnancy stage it is relevant for. This allowed the screens to dynamically render content items based off of their attributes. For example, when the content screen renders it checks if the 'relevant_for_roles' attribute of an item is equal to the user role and only displays if it is. Then, when that item is clicked, the display screen decides which rendering methods to use based off of the media type/s (Figure 13). Screens also do not read the data source (in this build the assets folder) directly. Every screen routes requests through the 'content repository', a common design technique known as the repository pattern [14]. It allows easily adding serving content from a remote source later, and supports changing what users are shown an item being a change to data and not to code. Supporting this data model meant that onboarding therefore had to collect a language, a role, and the stage of the pregnancy the user is either in, or supporting (Figure 16). The name and age of the user was also collected. The name allows users to be greeted personally on the home page and tell profiles apart. Age was collected in anticipation that some maternal health content may be

directed to certain age groups, such as resources which support teenage mothers. On launch, a user can add a new profile or pick an existing one (Figure 11). The application supports multiple profiles to address the reality that phones may be shared (UC4), and switching between profiles is available at any time by logging out of the current profile from the settings menu. This action returns the user to the profile picker.

4.4 Implementing Multiple Language Support

To support the same piece of content being available in both Sepedi and English, the data model stores them as different content items that share a canonical identifier. The application makes each pair one entry and the screen displays the one that matches the user's language preference. If an item exists in one language only, a screen will still display it, but mark it with a language badge to be explicit for users. Interface strings sit outside the content, and supporting multiple languages is implemented by using a library called i18n (internationalisation). This means that there is one file per language, and a screen asks for a label and not for the words it displays. The user switches language from the settings screen and every string re-renders without a restart. This implementation also allows strings displayed by the screen to be personalised by user role. For example, a family member reads about the pregnancy in the third person and the pregnant woman reads the same fact addressed to her (Figure 12). This directly addresses users wanting personalised, individually-relevant content rather than generic guidance (UC7), and prevents the app from feeling like it was only designed for the mother (UC9). All interface strings were translated by a home-language Sepedi speaker from the UL team.

4.5 Interface Design

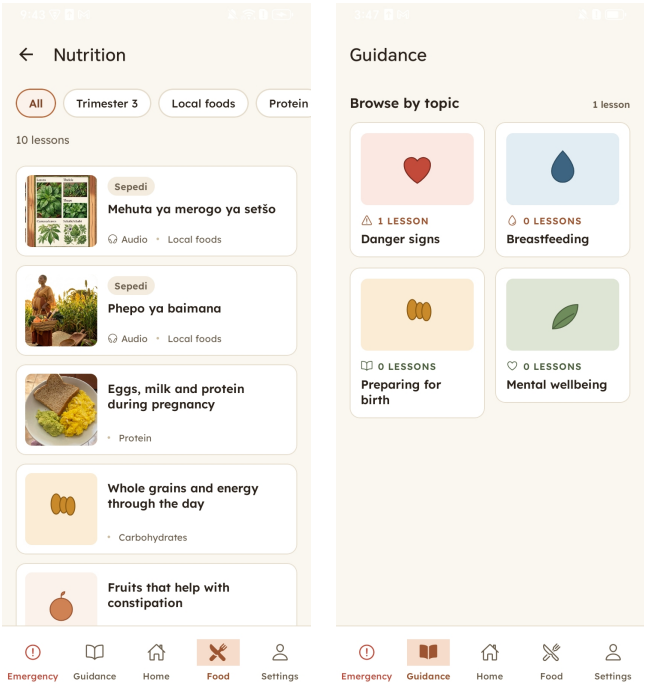
A UI was designed using WhatsApp's layout conventions as the reference, because Muthelo et al. [18] records WhatsApp as the most used platform among her participants, and it is also the platform MomConnect runs on [4]. Using these conventions aimed to mitigate the risk of deterring users from the application (UC6). Four of its conventions influence the interface the most: **(1)** a five-item bottom navigation bar (Figure 3), **(2)** a content row combining a thumbnail with a title (Figure 5), **(3)** filter chips above that list (Figure 6), **(4)** a settings screen of labelled rows, each with a leading icon and a trailing chevron (arrow) (Figure 4). The Lexend font was chosen as it was designed to improve reading fluency in low-proficiency readers [21]. The field work interaction plan also asked users to rank three UI layouts: search with a scrollable list, folders, and scrollable videos, so the application needed to display a scrollable list and folders. Scrollable videos were already represented by TikTok. To implement this, educational content was split into two top-level tabs: Food with scrollable rows, and Guidance with a two-column grid of folders (Figure 1). This split was chosen as delivering nutrition (food) content is the application's primary purpose and separating it from guidance prevents it from being diluted. The field work also wanted to test how easily users could find certain information across the three platforms. An emergency tab was added that was tinted red, represented by an exclamation mark, and reachable from anywhere in a single tap. It offers tap-to-dial contacts for an ambulance, the nearest clinic and the police. This UI choice was made to allow speed of identification and ease of access.

4.6 The Partner Demonstration

The field build was developed against these design decisions over four weeks, and it was demonstrated to Sepedi-speaking colleagues from the AMEN team. This was a valuable step as they had worked directly with the community of the target users. The session returned only two changes to be made before the field work: clearer iconography and more images.

5 Field Deployment and Evaluation

The field build was deployed over three consecutive days at three clinics in Limpopo: Evelyn Lekganyane, Makanye, and Dikgale. On the fourth day of the trip a symposium was held, and healthcare professionals also installed and tested the application. The application installed mostly on the participants' own mobile phones. Those without a suitable device were lent one of three extra devices with the app preinstalled. This allowed all participants to complete the activities. Each participant completed six tasks across the three platforms: the application, a maternal-health YouTube channel, and a TikTok channel containing essentially the same content. After using each platform, participants completed the same ten-item usability scale and answered written questions. A co-design session followed. In the first part, participants ranked the three platforms and three user-interface layouts. In the second, they assigned different types of content and features to their preferred platforms and mapped connections between community figures and interventions.



(a) Food **(b) Guidance**

Figure 1: Food and Guidance use different layouts.

In total, 51 people participated: 21 community health workers, 16 family decision-makers, and 14 pregnant women. The symposium was not a formal part of the study and its participants were not counted.

5.1 Installation and Distribution

The application was installed on 34 devices using one of three methods. Twenty-four installations used USB sideloading. Seven used Quick Share, a direct wireless transfer between Android phones. The remaining three used a local Wi-Fi hotspot serving an installer. These multiple installation methods were prepared to prevent any delays in the field work if one failed. The hotspot ran from a laptop, and a participant joined its network and scanned a QR code that opened the address serving the installer. This method was abandoned after three installations as participants needed substantial assistance to even connect to the WiFi hotspot. Sideloading the application was faster, and both a USB-C and a Micro-USB cable were taken to the field and each was tested beforehand to confirm it was a data cable and not charge-only. The 24 devices captured over USB came from six manufacturers and at least eighteen distinct models, every one of them budget or entry-level (Table 11). The application ran correctly across the range of devices and the interface adapted to the different screen sizes. Two attempted installations failed due to lack of space on the participants' phones. Two defects surfaced during the study. Opening trimester content from the home screen and pressing back navigated into the Food stack, and kept users stuck there. The application also sometimes crashed at the end of onboarding, but resumed correctly on reopening.

5.2 Field Work Observations

Two observations revealed usability issues in the UI design. (1) Many participants had difficulty identifying interactive elements. For example, some located a content row but did not tap it to open the content. Similar difficulties occurred during onboarding, where participants often did not realise that they needed to tap a form field to enter information and then select "Continue" to proceed. (2) Participants struggled to locate the Emergency tab, despite its design for rapid access (Section 4.5). When asked about this difficulty, many participants explained that they did not read the labels in the navigation bar and that the exclamation mark used to represent the Emergency tab did not signal a medical feature to them. Supporting these observations, the six facilitators produced field notes following the field work. These were used to compile application improvements, with supporting observations, for the receiving developer (Appendix A.6). The formal analysis of the task and co-design data was produced by Sarah-Jane Marais and was only available four weeks after the field work.

6 The Post-Fieldwork Build

6.1 Scoping the Post-Fieldwork Build

The post-fieldwork build was scoped around the requirements needed for the application to be used beyond the field work. These were drawn from requirements left unmet by the field build and from the maintainability requirements in Section 3.5. Downloading content for offline use (FR23), synchronising data to the backend (FR24) and content being changeable by a non-technical partner without a developer present (MR2) were prioritised. All three required implementing a remote backend, and downloading and synchronising also required a local store on the device. Changing content without a developer required implementing a content management system (CMS) with an interface built to serve non-technical content managers. The pilot date put implementing the remaining requirements and interface changes under time pressure, so the implementation of this sync system and CMS was completed between the field build and the handover. These were implemented so that the only requirements left for the receiving developer were predominantly interface level, and more modestly achievable considering his time and codebase experience constraints. Package deprecation monitoring (MR7) was also implemented in this build. As the project lead, Prof. Muthelo, is a non-technical partner, the monitoring mechanism was designed to provide her with a high-level overview of the application's status and enable her to refer any required maintenance to a technical partner. The back navigation defect observed in the field (Section 5.1) was also fixed in this build.

6.2 Backend Technology Selection

Supabase was used to implement the remote backend/database. It is an open-source backend-as-a-service platform built on PostgreSQL, and it was chosen because it could provide a free hosted database, authentication, and a storage bucket in one service. It was also chosen because its client is a JavaScript library, so it needs no native module in an Expo application. Its schema and data can be exported and recreated elsewhere, and its tooling can be driven by a coding assistant, which lowers the cost of maintaining the system or moving it for a developer who did not write it. SQLite was chosen for the on-device database, as it is lightweight and ships with Expo. Synchronisation was written rather than taken from a library, as the catalogue is small and content travels in one direction. PowerSync, WatermelonDB and Electric were also considered but rejected due to offering more than was necessary to implement the required system behaviour and would have added an extra dependency that needed monitoring and updating.

6.3 Implementation of the Content Management System

The CMS was built as a single-page web application in React 19. The mobile app is built with React Native, which uses React underneath, so a developer maintaining the application would not have difficulties maintaining the CMS. It uses Vite as its build tool and React Router for navigation. Content edited in the CMS needed to reach administrators as up-to-date data without a manual page refresh, so its data layer was built with TanStack Query to keep the dashboard's view of the database synchronised automatically. Content was also validated against its expected structure with Zod before being written to the database, so a malformed entry would be caught in the browser rather than surfacing later as a bug in the mobile app. The dashboard's login page is public. Administrators sign in with an email and password, authenticated through Supabase. Every other route is gated on an administrator flag, and the

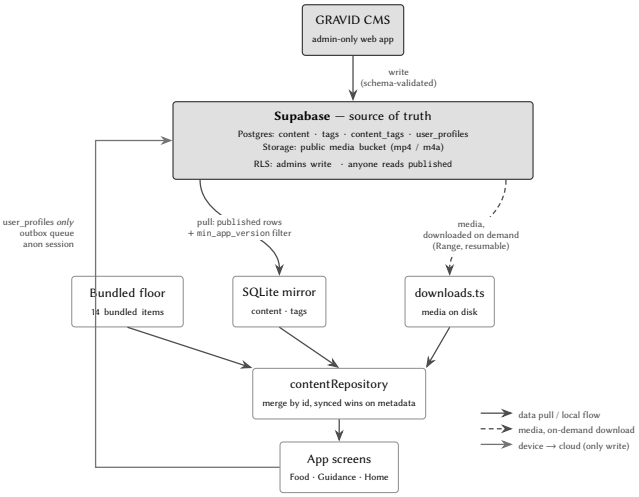


Figure 2: How a piece of content reaches a screen.

database enforces the same rule again, so a user without that flag cannot sign in or write anything. This status can only be granted by someone with access to the database. The CMS interface was modelled off of WordPress. It allows viewing the content, editing and adding content, and organising the top level categories and content tags. Figure 17 shows the UI. A content item is written as blocks that are dragged into order instead of as marked-up text, and supports the content items having any combination of text, audio, image, and video (FR6). A frame beside the editor renders the content as the user will see it, so the author does not need to publish to verify the formatting.

6.4 Implementation of Download on Demand and Synchronisation

Figure 2 shows the path a piece of content takes to a screen. Supabase holds the published copy and the CMS is its only writer, with read access granted to published items alone. Three copies are held on the device: the items bundled into the application, a mirror of the content and tag data, and the media the user has downloaded. The content repository merges them by identifier and prefers the synchronised data for titles and descriptions, so a device that has not synchronised still plays the content shipped with the application (FR22).

Synchronisation replaces the mirrored content and tag data on each run rather than retrieving what has changed, and takes only the rows marked published and permitted for the installed application version, so an older install is not sent content it cannot render. The device-local tables, the download state and the queue of pending writes, are therefore not constrained against the mirror, as rebuilding it would remove saved downloads and queued writes with it (Appendix A.15). The user profile is the only write that leaves the device, and it is held in that queue and sent under an anonymous session once a connection returns (FR24). Media is retrieved on demand (FR23, Figure 15), and a download resumes from the byte reached rather than restarting, as coverage is intermittent (UC2). A download is refused unless free space exceeds the remaining transfer (Figure 7), as device storage is a named barrier (UC3) and two installations failed in the field for lack of space (Section 5.1). Downloaded media is held against the device rather than against a profile, so each profile on a phone reports the same storage in use (Figure 14). Media is served from a public bucket rather than behind signed URLs, so the address of a long download cannot expire before it completes.

6.5 Implementing Package Deprecation Monitoring

To meet the requirement that deprecation notices reach an individual whose job it is to act on it and has the capacity to do so (MR7), a GitHub action was set up. A monthly job now runs in GitHub Actions that checks the application's dependencies and its framework version and emails a one-page status to the project lead (Figure 8). It runs on GitHub's servers, so no machine of anyone's has to be on for the check to happen. The status is one of three phrases in plain language, the technical detail sits below a section addressed to the non-technical partner, and the action asked for is to forward the mail to whoever holds the technical contract.

7 The Handover

Eight sessions were planned with the receiving developer, to hand over three things: (1) the source code and the accounts the application runs on, (2) the requirements that remained unmet, and (3) the changes the field work showed were needed before a wider pilot. The post-fieldwork build was ready before the first session. Due to the imminent pilot deadline, the handover process started one week after the fieldwork, before the field notes had been analysed. **Session one** covered cloning the GitHub repository, installing the dependencies, explaining the codebase, and reviewing the refined application requirements. Before the handover, the project lead had provided him with the original PRD. During the post-fieldwork handover, he was introduced to the formalised functional requirements (Table 2) and shown how they traced back to the PRD (Appendix A.3). The requirements rejected during the requirements analysis, and the reasons for their rejection, were also explained to prevent their implementation. Finally, a discussion about what was observed in the field work took place (Section 5). Four tasks were handed over that were achievable and would help to teach him the codebase: (1) begin an analytics setup (FR25), (2) calculate a due date from the date of the last period at onboarding, (3) move to square icons, using the Guidance tab as the reference but larger, with photographs or several icons that convey a meaning together, and (4) give every clickable item a chevron and a clearer lift from the background. Tasks three and four came from what was observed in the field before any field notes had been written up (Section 5.2). **Session two** took place three days later, and the four tasks were complete. The receiving developer was working with an agentic coding assistant and used it to have the architecture explained to him and to iterate, and he attributed much of his speed to it. During this session, three requirements from the PRD that had not yet been implemented were handed over: two-sided myth-busting cards (FR8), reminders and the settings that control them (FR17, FR18), and a question-and-answer feature (FR20, FR21).

Session three ran a week later. The field notes from the facilitators were available by then and a recommendations document was compiled from them (Appendix A.6). The receiving developer also demonstrated the application's current state. It was observed that the requirements from the previous session had been implemented. **Session four** used the first available results of the formal analysis of the field data, available by then from Sarah-Jane Marais. The receiving developer was taken through the findings even where he had already made the required change in an earlier session, so that the reasoning behind a change was clear to him, and not only the change necessary itself (MR4). The application was demonstrated again. It was observed that meal logging (FR19) and adjustable font size and playback speed (FR14, FR15) had been implemented by his own agency. However, none of the changes requested by the recommendations document were observed at this point. The requirements handed over as still outstanding at the end of that session were subtitle tracks on video and audio (FR13) and a low-literacy mode (FR16). Changes identified during the fieldwork were to take priority, so these requirements were assigned for implementation before the pilot only if time allowed, or after the pilot if necessary. A discussion was also had regarding encrypting user information before the Q&A feature was deployed, and migrating to a different database when the project reached a stage where rapid expansion in the number of users was anticipated. The AMEN project lead was granted full access to the Supabase project and the GitHub repository during this handover period as well. Environment variables or keys did not need to be handed over, and the reasoning is outlined in Appendix A.17. The receiving developer was not respondent to communication after the fourth session.

8 Results

8.1 Non-functional Requirements

Measurements were taken on the Redmi 9A device and Appendix A.9 and Appendix A.10 give the methods and readings. Seeing an interactive screen takes a median of 1452.5 ms after starting the application, against a target of 3 seconds (NFR5). Two Samsung Galaxy A03 Core handsets with 2 GB of RAM successfully installed and ran the application alongside the other tested devices (NFR4, Appendix A.8). Two NFRs were not met: (1) The installed package is 64.5 MB, exceeding the 50 MB limit (NFR3). A justification for not strictly meeting this requirement can be found in Appendix A.9. (2) Encrypting on-device data (NFR8) was discussed with the receiving developer but cannot be verified as met (Section 7). Six requirements could not be measured within the scope of this paper. Support for 5000 concurrent users (NFR11). NFR1 sets no figure to measure encoding against. Touch feedback and target size (NFR6, NFR7) were not measured. Compliance with GDPR and POPIA (NFR9) was not reviewed, though backend access runs under row-level security and all traffic to it over HTTPS, and anonymised analytics (NFR10) will have to be implemented by the receiving developer.

8.2 Functional Requirements

The field build included onboarding and multiple profiles on a device (FR2 to FR5), the content delivery customised by language, role and category (FR6, FR7, FR10 to FR12), and offline operation (FR22), as Section 4 describes. The post-fieldwork build added the two offline requirements that need a backend (FR23, FR24) and moved content authoring outside the code (Section 6). Eleven outstanding requirements were given to the receiving developer (Section 7). It was not possible to formally verify which of these were implemented because the source code could not be accessed, as his changes were not pushed to the remote repository. It was however observed remotely that eight of the requirements handed over had been implemented (Section 7). The AMEN team also reported that the build was used in the field for a wider pilot. FR13 and FR16 were not observed as implemented. Recording or uploading a testimonial (FR9) was not implemented. Because recorded testimonials would be shared with other users, they would require moderation before publication, and neither team had assigned responsibility for this task. This left a requirement that directly addresses UC8 (users' stated need for two-way, interactive communication) unresolved. Whether it will be implemented is left as a decision for the AMEN team.

8.3 Automated Tests and Device Verification

Automated tests cover the content model. Download on demand and updating content from the dashboard was verified by hosting the CMS on a local machine and using the Redmi 9A. Appendix A.12 gives the coverage of the testing and the gaps.

8.4 Maintainability Requirements

Four of the seven are met by what Sections 4 to 7 describe. The application is written in a language a successor is likely to know (MR1), its repository is remote and under version control (MR3), the CMS changes content items, categories and tags without a release and without a developer (MR2), and the AMEN project lead has administrator access to both the database and the repository (MR5). The observation behind MR1 is the handover. The receiving developer had not seen the codebase before the first session, had it running by the end of that session and completed four tasks in it within three days. Deprecation monitoring (MR7) is met for packages and not for the platform. The monthly job reaches the project lead with the state of the dependencies and the action to take, while the store deadline notices Google and Apple send go to the account holder alone and force a rebuild, so those still have to be forwarded by hand. The two requirements that depend on a recipient are partly met. The overlap between the outgoing and the incoming developer ran to four of the eight planned sessions (MR6). The reasoning behind the code was recorded and reached him through conversation in those four sessions and through documentation (MR4).

9 Discussion

9.1 A Familiar Layout Did Not Make Affordances Clear

Medhi et al. [17] recommend designing for low digital literacy with fewer controls and larger targets. Till et al. [25] found that participants preferred technology they already understood. This project applied the latter finding by using WhatsApp's layout conventions with the aim of increasing usability, but the field work showed the limits of this design choice. Participants found content rows but

did not know to tap them, and often did not recognise that form fields required input (Section 5.2). This result implies that copying a layout does not transfer users' understanding of the affordances of its components. The emergency tab is an interesting case, because it was designed for speed of identification and failed on identification. Coleman et al. [5] argue that first-language speakers must be involved in creating content from the start, and this observation implies that this argument should be extended from content to iconography.

9.2 Continuity Fails Independently of Documentation

None of the three custody failures behind MR5 (Section 3.5) is a failure of documentation. The 2021 MM project's codebase was commented, in a repository and with README files, and its credentials still sat with a graduate two years on. What each case lacked was a place in the institution for the artefact to be held, which is what the access recommendations in Section 2.3 address. Conger [7] finds this at programme scale across the Living Labs in South Africa, where technology is implemented and left with local caretakers without continued development. One of her cases is close to this project's stated destination, a patient health system that reached several clinics and a regional hospital and was discontinued when the municipal health department would not take responsibility for running it. Her answer is to identify several people and their backups early in a project's life and train them, because a single manager continuing the work is unlikely. Foster et al. [13] reach the institutional version of the same point from an undergraduate project handed across four teams and two cohorts, that universities should hold policies on ownership of student and staff collaborations instead of settling ownership case by case. All three recommendations are about who holds the artefact, and a better-documented codebase does not satisfy any of them. Fehr et al. [11] also identify clear ownership as a condition for maintainability, but describe it as something established at handover. The failure from MM this project aimed to avoid occurred in a year when there was no receiving developer and therefore no handover, showing that custody must remain assigned between handovers as well.

9.3 Limitations of Documentation For Transferring Knowledge

The reasoning behind the departures from the PRD and every recommended interface change from the field work reached the receiving developer (Section 7). That is what Jansen and Bosch [16] and Sibeko [22] ask for, and the retrospective cautions against seeing this as the fix for maintainability issues. The 2021 MM group met the standard Fehr et al. [11] set but still failed because no group took the project on the following year. Densmore's account of the one year MM was maintained under a formal agreement is that everything was documented so that it could be handed to another student and there was no student to hand it to. It again failed after this ended. The quality of the documentation therefore did not decide either outcome. Both failures were a product of a lack of a recipient with the incentive and capacity to maintain it. The overlap Fehr et al. [11] recommend is unavailable at the honours seam. A project ends in October, the next group does not exist until March and, by Densmore's account, does not begin reviewing the previous work until April or May. This project had an overlap only because the receiving developer was from a different institution and not restricted by the academic calendar. Even with the advantage of this overlap, only half of the planned handover sessions occurred. Foster et al. [13] recommend that the outgoing and incoming teams meet on several occasions, that students from the earlier team act as mentors, and that the handover be made part of the marking criteria. The eight planned sessions were intended to implement the first two recommendations. The third is institutional and was outside this project's control. Conger [7] shows that knowledge transfer is rarely planned in ICT4D projects. Of more than 900 papers published between 2000 and 2010, only seventeen percent discussed training or communication, while knowledge management was largely absent from the literature. None of the projects she reviewed treated knowledge transfer as a planned activity. This lack of planning is also visible in the retrospective. The 2021 MM developer's reason for keeping screen brightness low because babies woke at night appeared in neither the code nor that year's papers, and was recovered only when she stated it in an interview five years later. The 2023 BBP group removed a clinic map feature from the platform that they understood to be a previous team's unjustified assumption. In reality, Coleman [6] records it as a feature requested by the community. A 2023 BBP developer was asked what reasoning from previous groups he was able to recover, and said that "everything we were able to discover was purely from reading the papers", with no access to further documentation or recordings. This implies that recording reasoning behind code and design is not enough to transfer the knowledge to a receiving group. The method of transferring the reasoning and ensuring its receipt appears equally important.

10 Limitations

The handover involved one outgoing developer and one receiving developer. There was no attempt to hand over the application to another developer using a different protocol. Therefore, the handover as it occurred cannot establish whether including rationale behind tasks and having live handover meetings improved the recipient's work. The project's ethics clearance did not permit a direct assessment of his understanding behind the code and the changes he needed to make, so knowledge transfer could only be evaluated through the changes he completed. Only four of the eight planned sessions took place. The reason for the communication failure and the remaining sessions not occurring cannot be confirmed, although the challenges of working remotely across two institutions may have contributed. The demonstrations during the handover sessions, along with the AMEN team's report that the version of the application built by the receiving developer was used successfully in the pilot, are the only reportable evidence of handover success. The download layer, the synchronisation and the CMS were built after the field study, so no target user has evaluated their usability. This has been communicated as future work to the AMEN project lead. All non-functional measurements were taken on the test device with 2.7GB of RAM, against a specified minimum of 2GB, so the readings are likely better than they would

be on the baseline device. Another limitation is that only the devices that had the application installed over USB were recorded. Appendix A.8 covers twenty-four of the thirty-four devices that received the application. The retrospective rests on four purposive interviews and on the written records of projects that have ended, and it is not a thematic analysis. Findings are reported where two or more accounts converge, or where one account is testable against another's case. The honours papers describe each system as it stood at submission, and by supervisor Densmore's account, system changes and deployments often occurred after the paper deadline. This means that a successor's account of the system they inherited is likely the most accurate, and only two of them were interviewed for this paper. The small interview size and the length of time that has passed since the interviewees were involved in their respective projects limits the strength of the findings. Every project in the retrospective also predates agentic coding assistance, so the challenges of working with an inherited system may no longer be the same as they were for them. The field work did not record whether participants failed or succeeded to complete the application navigation tasks. This means that the interface usability issues were only recorded by facilitators' observations. None of the participants wrote detailed notes on which components of the interface they struggled to use. Interestingly, eighteen of the 51 participants used difficulty language in their written answers, but thirteen of them still scored the application at twenty-four or more out of thirty, and independent use scored almost identically low across all three platforms, so difficulty unaided is a property of this population and not of this application. That left the interface changes resting on observation and not on the quantitative data. The workshops ran with four English-speaking facilitators and two Sepedi speakers, so it was not always possible to tell whether a participant had understood an activity, and nothing in the worksheets records which facilitator ran which session. Late delivery of content was the most critical risk identified for the project and it materialised. The written nutrition and trimester material was compiled from World Health Organization and Mayo Clinic guidance, drafted into the required format by a large language model, and translated and reviewed by a UL team member shortly before the field work, instead of being supplied by the AMEN team, and the Sepedi videos came from Stanford's Digital Medic library. The application was therefore evaluated with content that is not the final content it will deliver.

11 Future Work

The outcome of emphasising maintainability in this project could not be reported due to the timeline of this paper. Observing and reporting what has happened to the application a year from now could provide valuable insights for similar projects. Interviewing the receiving developer and the project lead again will be necessary to establish whether the application and CMS are still running and still maintained. The strength of the findings of this paper should also be tested by examining multi-year projects in other departments. The interviews also returned material that could not be reported on in the scope of this paper. There is potential to write a paper with insights on why a research programme does not reward maintaining software, on how a research group might improve continuity between cohorts, and on the demonstrations a non-technical owner needs where a developer needs a document, and on the technical fluency a partner organisation accumulates across cohorts.

12 Conclusion

GRAVID-AMEN was developed through three versions. The field build was installed on 34 devices and evaluated with 51 participants across three clinics in Limpopo. The field work showed that using a familiar layout did not make the affordances of its components evident. The post-fieldwork build added the backend, download on demand, synchronisation and the CMS. After the application was handed over, the receiving developer demonstrated a further version that the AMEN team reported was used in a wider pilot in August 2026. Aim 1 was partly met. The field and post-fieldwork builds delivered the functional requirements described in Sections 4 and 6, and met the tested launch-time and device-compatibility requirements. The installed package exceeded its size limit, local data encryption could not be verified, and six non-functional requirements were not assessed. Eleven outstanding functional requirements were handed to the receiving developer. Eight were observed as implemented during a demonstration, but could not be verified against the source code because his changes were not pushed to the remote repository. Aim 2 was also partly met. Four of the seven maintainability requirements were met: the codebase uses a language a successor is likely to know, the repository is remote and under version control, the CMS allows content to be changed without a developer, and the AMEN project lead holds administrative access. The transfer of design rationale and the overlap between developers were partly completed because only four of the eight planned handover sessions took place. Deprecation monitoring was implemented for application packages, while platform notices still depend on manual forwarding. The retrospective shows that documentation and version control alone do not preserve continuity. GRAVID-AMEN addressed specific failures from the earlier projects by transferring administrative access, separating content management from development, recording the reasoning behind decisions and routing dependency warnings to the project lead. This project did have a receiving developer, and the changes he demonstrated show that development continued during the handover. They do not establish whether the recorded rationale improved his work or whether maintenance continued after the pilot. The conclusion is therefore narrower: a maintainable artefact and an initial handover can support a transition, but continued maintenance depends on responsibility remaining assigned after that transition and on maintenance warnings reaching someone with the responsibility and capacity to act.

References

[1] Frank Adusei-Mensah, Livhuwani Muthelo, Mxolisi Welcome Ngwenya, MH Mphasha, and Jussi Kauhanen. 2025. Digital health interventions for pregnant women and mothers with under 5-year-olds in low- and middle-income countries: a scoping review. *Global Health Journal* 9, 3 (2025), 113–123. doi:10.1016/j.glohj.2025.06.008

[2] Ayomikun S. Akinseinde, Joke A. Badejo, and Rejoice L. Malgwi. 2016. GRAVID: An Indigenous m-Health Tool for Smart & Connected Communities. In *2016 Future Technologies Conference (FTC)*. IEEE, San Francisco, CA, USA, 1331–1334. doi:10.1109/FTC.2016.7821776

[3] Tariro Banganayi. 2023. You and Your Baby 2.0: Refactoring and Evaluating an Early Childhood Development Resource Management System. Computer Science Honours Final Paper, University of Cape Town.

[4] Peter Barron, Justin Peter, Amanda E. LeFevre, Jane Sebidi, Mia Bekker, Rachel Allen, Andrew N. Parsons, Peter Benjamin, and Yogan Pillay. 2018. Mobile health messaging service and helpdesk for South African mothers (MomConnect): history, successes and challenges. *BMJ Global Health* 3, Suppl 2 (2018), e000559. doi:10.1136/bmjgh-2017-000559

[5] Toshka Coleman, Sarina Till, Jaydon Farao, Londiwe Shandu, Nonkululeko Khuzwayo, Livhuwani Muthelo, Masenyani Mbombi, Mamare Bopape, Alastair van Heerden, Tebogo Mothiba, Shane Norris, Nervo Verdezoto, and Melissa Densmore. 2023. Reconsidering Priorities for Digital Maternal and Child Health: Community-centered Perspectives from South Africa. *Proc. ACM Hum.-Comput. Interact.* 7, CSCW2 (2023), 1–31. doi:10.1145/3610081

[6] Toshka Lauren Coleman. 2023. *Co-designing Community-based Digital Innovations for Maternal and Child Health (MCH).* Master of Science dissertation. University of Cape Town, Cape Town, South Africa.

[7] Sue Conger. 2013. Knowledge Management in LLiSA ICT4D Projects. In *Proceedings of the South African Institute for Computer Scientists and Information Technologists Conference (SAICSIT '13).* Association for Computing Machinery, New York, NY, USA. doi:10.1145/2513456.2513464

[8] Janan Janine Dietrich, Lerato Tsotetsi, Thenjiwe Dubazane, Gugulethu Tshabalala, Boitumelo Maimela, Martin Weiss, and Mamakiri Mulaudzi. 2023. A qualitative study to explore strategies to improve the Road to Health Application for maternal and child health outcomes in South Africa. *Frontiers in Digital Health* 4 (2023), 1094754. doi:10.3389/fdgth.2022.1094754

[9] Sarah Dsane-Nsor, Aakash Gautam, Carolina Fuentes, Gideon Kwesi Toffah, Nervo Verdezoto, Yaseen Joolay, and Melissa Densmore. 2025. Navigating Sociocultural Tensions in Prenatal Care: Opportunities for Digital Health from Diverse Stakeholders in Techiman, Ghana. *Computer Supported Cooperative Work (CSCW)* 34, 2 (2025), 741–784. doi:10.1007/s10606-025-09521-5

[10] Jaydon Farao. 2023. Digital Storytelling for the Support of Maternal and Child-wellbeing Through Community-based Co-design. In *Extended Abstracts of the 2023 CHI Conference on Human Factors in Computing Systems (CHI EA '23).* ACM, New York, NY, USA, 1–5. doi:10.1145/3544549.3577050

[11] Jörg Fehr, Christian Himpe, Stephan Rave, and Jens Saak. 2021. Sustainable Research Software Hand-Over. *Journal of Open Research Software* 9 (2021), 5. doi:10.5334/jors.307

[12] Beatriz Félix, Cristiana Braga, Xolani Ntinga, Sarina C. Till, Leina Meoli, Alastair Van Heerden, Ricardo Melo, Nervo Verdezoto, Melissa Densmore, and Francisco Nunes. 2024. Understanding How Parents Deal With the Health Advice They Receive: A Qualitative Study and Implications for the Design of Message-based Health Dissemination Systems for Child Health. In *Proceedings of the 2024 ACM Designing Interactive Systems Conference (DIS '24).* ACM, New York, NY, USA, 1–17. doi:10.1145/3643834.3661504

[13] David Foster, Filippo Gilardi, Paul Martin, Wei Song, Dave Towey, and Andrew White. 2018. Students as Co-producers in a Multidisciplinary Software Engineering Project: Addressing Cultural Distance and Cross-cohort Handover. *Teachers and Teaching* 24, 7 (2018), 840–853. doi:10.1080/13540602.2018.1486295

[14] Martin Fowler. 2002. *Patterns of Enterprise Application Architecture.* Addison-Wesley, Boston, MA.

[15] GeeksforGeeks. 2025. Top 10 Mobile App Development Frameworks. https://www.geeksforgeeks.org/blogs/top-mobile-app-development-frameworks/. Accessed 6 September 2026.

[16] Anton Jansen and Jan Bosch. 2005. Software Architecture as a Set of Architectural Design Decisions. In *Proceedings of the 5th Working IEEE/IFIP Conference on Software Architecture (WICSA 2005).* IEEE, 109–120. doi:10.1109/WICSA.2005.61

[17] Indrani Medhi, Aman Sagar, and Kentaro Toyama. 2007. Text-Free User Interfaces for Illiterate and Semiliterate Users. *Information Technologies and International Development* 4, 1 (2007), 37–50. doi:10.1162/itid.2007.4.1.37

[18] Livhuwani Muthelo, Masenyani Oupa Mbombi, Mamare Adelaide Bopape, Tebogo M. Mothiba, Melissa Densmore, Alastair van Heerden, Shane A. Norris, Nervo Verdezoto Dias, Paula Griffiths, and Nicola Mackintosh. 2023. Reflections on Digital Maternal and Child Health Support for Mothers and Community Health Workers in Rural Areas of Limpopo Province, South Africa. *Int. J. Environ. Res. Public Health* 20, 3 (2023), 1842. doi:10.3390/ijerph20031842

[19] Richard Pankomera and Darelle van Greunen. 2018. A Model for Implementing Sustainable mHealth Applications in a Resource-Constrained Setting: A Case of Malawi. *The Electronic Journal of Information Systems in Developing Countries* 84, 2 (2018), e12019. doi:10.1002/isd2.12019

[20] Justin Peter, Peter Benjamin, Amanda E. LeFevre, Peter Barron, and Yogan Pillay. 2018. Taking digital health innovation to scale in South Africa: ten lessons from MomConnect. *BMJ Global Health* 3, Suppl 2 (2018), e000592. doi:10.1136/bmjgh-2017-000592

[21] Bonnie Shaver-Troup and Thomas Jockin. 2018. Lexend: a variable font family designed to aid reading proficiency. https://github.com/googlefonts/lexend. Accessed 1 September 2026.

[22] Lwazi Sibeko. 2023. Re-engineering a Platform for Managing Early Childhood Development Resources. Computer Science Honours Final Paper, University of Cape Town.

[23] Deborah Talbot. 2024. *Milk Matters 4.0: Bridging Milk Donor, Staff and Student Needs towards a Purposeful and Maintainable System.* Master's thesis. University of Cape Town.

[24] Deborah Talbot and Melissa Densmore. 2023. Milk Matters 4.0: Challenges in Deploying University-led Mobile Application Development for Small NGOs. In *Poster presented at the ACM SIGCAS/SIGCHI Conference on Computing and Sustainable Societies (COMPASS '23).* Cape Town, South Africa. https://pubs.cs.uct.ac.za/id/eprint/1555/

[25] Sarina Till, Jaydon Farao, Toshka Lauren Coleman, Londiwe Deborah Shandu, Nonkululeko Khuzwayo, Livhuwani Muthelo, Masenyani Oupa Mbombi, Mamare Motlhatlhedi, Molebogeng Motlhatlhedi, Gugulethu Mabena, Alastair Van Heerden, Tebogo Maria Mothiba, Shane Norris, Nervo Verdezoto, and Melissa Densmore. 2022. Community-based Co-design across Geographic Locations and Cultures: Methodological Lessons from Co-design Workshops in South Africa. In *Proceedings of the Participatory Design Conference 2022 - Volume 1 (PDC '22).* ACM, New York, NY, USA, 120–132. doi:10.1145/3536169.3537786

[26] Sarina Till, Mirriam Mkhize, Jaydon Farao, Londiwe Deborah Shandu, Livhuwani Muthelo, Toshka Lauren Coleman, Masenyani Mbombi, Mamara Bopape, Sonja Klingberg, Alastair van Heerden, Tebogo Mothiba, Melissa Densmore, and Nervo Xavier Verdezoto Dias. 2023. Digital Health Technologies for Maternal and Child Health in Africa and Other Low- and Middle-Income Countries: Cross-disciplinary Scoping Review With Stakeholder Consultation. *J Med Internet Res* 25 (2023), e42161. doi:10.2196/42161

[27] Aurene Wilford, Sifiso Phakathi, Lyn Haskins, Ngcwalisa A. Jama, Ntokozo Mntambo, and Christiane Horwood. 2018. Exploring the care provided to mothers and children by community health workers in South Africa: missed opportunities to provide comprehensive care. *BMC Public Health* 18 (2018), 171. doi:10.1186/s12889-018-5056-y

A Supplementary Information

A.1 Original PRD Functional Requirements

A.2 Original PRD Non-Functional Requirements

Table 5: Functional requirements as listed in the original Product Requirements Document.

Functional Requirements
User Onboarding
Registration via phone number (South Africa format: +27)
Language selection: English and Swahili
Role-based profile creation (Pregnant Woman, CHW, Family Member, Professional)
Pregnancy stage input (weeks or trimester)
Optional: Family members can create profiles on behalf of pregnant women
Nutrition Education Modules
Culturally tailored learning units with text + voice narration
Illustrated infographics with minimal text
Audio-first explanations for low-literacy users
Topics: balanced diet, local food options, safe supplements, myths & facts
Professional connections: nutritionists, midwives, farmers associations
Myth-Busting Section
Interactive myth vs. fact cards with audio-visual explanations
Stories debunking local food taboos
Community testimonies from trusted elders
Trusted Voices Feature
Pre-recorded messages from nurses, CHWs, and elder women
Content segmented by theme: supplements, cravings, hygiene, family planning
Video and audio formats with subtitles
Reminders and Alerts
Meal logging functionality with gentle reminders
Daily learning nudges
Customizable ANC visit prompts based on pregnancy stage
Supplement intake reminders
Peer Stories and Testimonials
Video, audio, and text-based community testimonials
Categorized by: food beliefs, pregnancy stage, coping strategies
User-generated content option (Phase 2)
Family Mode
Shared content mode for partners and elders
Respectful, simplified education modules
Visual and audio-first formats
Guidance on supporting pregnant women
Professional Q&A (Phase 2)
Mentor-facing resource hub with FAQs
Ask-a-nurse voice note feature
Optional two-way communication integration
Offline Content Access
Offline-first architecture: All core features function without internet connectivity
Pre-downloadable content bundles by module
Low-bandwidth image and audio formats
Sync capability when connectivity is available
App Settings and Preferences
Language switcher (English/Swahili)
Playback speed control for audio content
Font size adjustment
Reminder and notification toggle
AI Content
Use current AI tools to generate short videos with local avatars in local languages (and English) in a folklore type of setting to explain food sourcing, processing, preparation and storage.
Use AI to generate comic booklets with short stories, flash cards

Table 6: Non-functional requirements as listed in the original Product Requirements Document.

Non-Functional Requirements
Performance
Network support: Optimized for 2G/3G environments
Module size: <2MB per content bundle
Installation size: <50MB lightweight package
Device compatibility: Runs on low-spec Android devices (<2GB RAM)
Load time: <3 seconds for content modules
Usability
Designed for low-literacy users with large icons and voice assist
Intuitive navigation requiring minimal training
Touch-optimized interface with clear visual feedback
Accessibility features: adjustable font sizes, high contrast mode
Security
Phone-based authentication (SMS OTP)
Secure local storage with encryption
Data privacy compliance: GDPR and POPIA (South Africa)
Anonymous usage analytics
Scalability
Backend architecture extensible for additional languages
Modular content management system for regional adaptation
Cloud-based content delivery with local caching
Supports 5,000+ concurrent users in pilot phase

A.3 Functional Requirements

Table 7: Each reworked functional requirement and its source in the original PRD. The requirement label links to its row in Table 2 (Section 3.3).

Requirement	Source in the PRD
FR1: A user can register with a phone number, verified by SMS one-time passcode.	"Registration via phone number" and "Phone-based authentication (SMS OTP)" from the security NFR list, folded in as the mechanism for the phone-number path.
FR2: A user can select a display language at registration and change it later in settings.	Merges "Language selection" (onboarding) and "Language switcher" (settings).
FR3: A user can select a role at sign-up (pregnant woman, partner/family member, elder, CHW, nurse).	"Role-based profile creation."
FR4: The application captures a pregnancy stage or due date on the profile.	"Pregnancy stage input." Feeds stage-relevant reminders (see "ANC visit prompts by pregnancy stage" below) and content relevance, but that link is left out of the statement itself.
FR5: A user can create and manage more than one linked profile (e.g. a CHW managing client profiles).	"Family profiles on behalf of others."
FR6: The application supports content items in any combination of text, audio, image and video.	Replaces "text and voice narration," "illustrated infographics," "audio-first explanations" (Nutrition Modules), the format half of "video and audio formats with subtitles" (Trusted Voices), "video, audio and text testimonials" (Peer Stories), and "visual and audio-first formats" (Family Mode).
FR7: The application supports tagging content with one or more categories (topic, theme, stage, belief, coping strategy).	Replaces "Topics: balanced diet, local foods, supplements, myths and facts," "Content segmented by theme," "Categorised by belief, stage, coping strategy."
FR8: The application supports a reusable two-sided card type, a statement on the front and a reveal action on the back.	"Interactive myth vs fact cards."
FR9: A user can record or upload their own testimonial (text, audio or video).	"User-generated content."
FR10: The application presents interface text and, where available, content in the selected language.	Same as FR2. Depends on FR2.
FR11: The application customises feature and content access by profile role.	Merges with Family Mode's "Shared content mode for partners and elders," the same role-gated access applied to a family/elder role. Depends on FR3.
FR12: A user can select a category to view the content tagged with it.	Same as FR7. Depends on FR7.
FR13: The application supports caption/subtitle tracks on video and audio.	"Video and audio formats with subtitles."
FR14: A user can adjust the font size for text content.	"Font size adjustment."
FR15: A user can adjust the playback speed of audio and video.	"Playback speed control."
FR16: A user can select a low-literacy mode that defaults to audio and image over on-screen text.	"Audio-first explanations for low-literacy users."
FR17: A user can enable or disable each reminder category independently.	"Reminder and notification toggle," placed next to the scheduler it controls.
FR18: The application schedules reminders on a fixed interval (a daily learning nudge), a profile date or stage (ANC visit prompts by pregnancy stage), or a user-logged event (supplement intake).	"Daily learning nudges," "ANC visit prompts by pregnancy stage," "Supplement intake reminders," and the reminder half of "Meal logging with reminders." One scheduling engine with three trigger types, rather than four separate features.
FR19: A user can log meals or food intake against their profile.	"Meal logging with reminders," logging half; the reminder half is FR18c.
FR20: A user can message a professional account (e.g. a nurse) and receive a reply.	"Professional Q&A: ask-a-nurse, two-way communication," end-user side. "Resource hub" is not treated as separate, browsing content by category is FR7/FR12.
FR21: A professional account can view and reply to messages from users.	Same source line, professional side. "Two-way communication" only holds if both actors have a defined capability; the professional's was not stated separately in the original list, so this fills the other half rather than adding new scope.
FR22: The application keeps bundled or downloaded content viewable offline, and needs no connection to display the entry screen, switch language, or switch profile.	"All core features function without connectivity," narrowed to what is actually testable. Entry screen display, language switching (FR2) and profile switching (FR5) all operate on data already on the device. Two-way messaging (FR20/FR21) is excluded, since it requires a live connection by definition.
FR23: A user can download a content module for offline use.	"Pre-downloadable content bundles by module."

continued on next page

Table 7 continued

Requirement	Source in the PRD
FR24: The application synchronises offline data to the backend once connectivity returns.	"Sync when connectivity is available," with the subject flipped for consistency.
FR25: The application collects usage analytics.	Implied by "Anonymous usage analytics" (security list); the anonymisation constraint is NFR10, this states the underlying collection that constraint applies to. What is tracked and who sees it is not specified in the source.

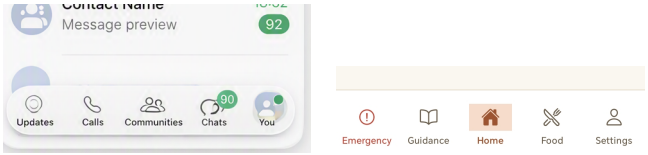
A.4 Non-functional Requirements

Table 8: Each reworked non-functional requirement and its source in the original PRD. The requirement label links to its row in Table 3 (Section 3.4).

Requirement	Source in the PRD
NFR1: The application encodes and delivers video and audio within a bitrate/file-size threshold suitable for 2G/3G networks.	"Low-bandwidth image and audio formats" (original FR list) and "Network support: Optimized for 2G/3G environments" here. "Network support" as phrased is not independently testable, "optimized" is not measurable; this NFR plus NFR2–NFR4 below are its testable decomposition, so it is not restated as its own item.
NFR2: Each content bundle stays under 2MB.	"Module size: <2MB per content bundle."
NFR3: The installed package stays under 50MB.	"Installation size: <50MB lightweight package."
NFR4: The application runs on Android devices under 2GB RAM.	"Device compatibility: Runs on low-spec Android devices (<2GB RAM)."
NFR5: The application displays its entry screen within 3 seconds of opening from the home screen.	"Load time: <3 seconds for content modules," reinterpreted as app launch time rather than in-app content loading. Entry screen display is a core offline capability (FR22), so this is measured on the reference low-spec device from NFR4 only, no network condition applies.
NFR6: The application shows a visible state change on touch.	"Touch-optimized interface with clear visual feedback," the only testable part of that line. Minimum touch target size is NFR7.
NFR7: Interactive elements have a minimum touch target of 48x48dp.	"Touch-optimized interface with clear visual feedback" and "large icons" (usability list); no size was given in the source, so this follows Android's own accessibility guideline (Material Design Accessibility / Android Accessibility Help, minimum 48x48dp) rather than an invented number.
NFR8: Data stored locally on the device is encrypted.	"Secure local storage with encryption."
NFR9: The application complies with GDPR and POPIA data protection requirements.	"Data privacy compliance: GDPR and POPIA (South Africa)."
NFR10: Usage analytics under FR25 are anonymised, with no data stored or transmitted in an identifiable form.	"Anonymous usage analytics," listed under Security alongside encryption and GDPR/POPIA compliance in the original list, read as a privacy constraint on analytics rather than a new analytics feature. Depends on FR25. Moved here from Scalability, where it was misplaced by category.
NFR11: The application supports at least 5,000 concurrent users in the pilot phase.	"Supports 5,000+ concurrent users in pilot phase."

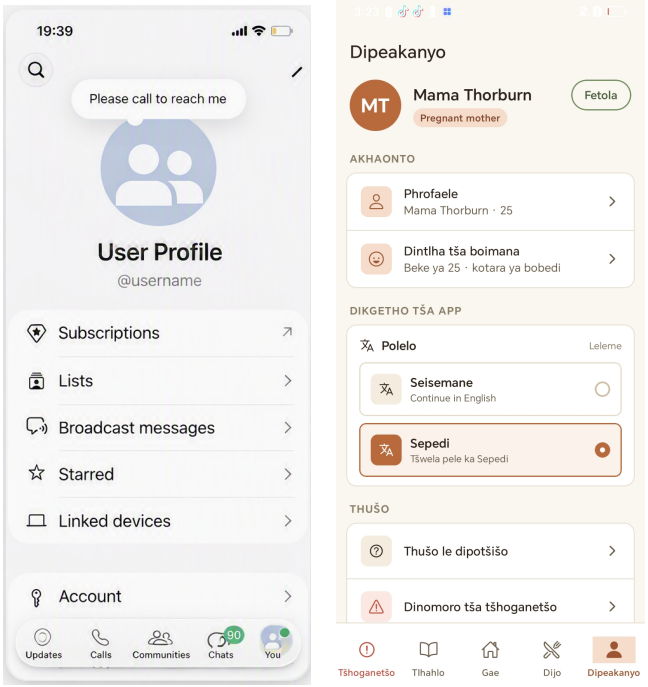
A.5 Interface Comparison with WhatsApp

Section 4.5 takes WhatsApp's layout conventions as the reference for the interface, because Muthelo et al. [18] records it among the platforms this audience already uses. Figures 3, 4, 5 and 6 set the tab bar, the settings screen, the content list and the filter chips above it against the WhatsApp screenshots that informed the design, captured on a personal phone with contact names anonymised.



(a) WhatsApp (b) GRAVID-AMEN

Figure 3: The bottom tab bar. Both use five icons with a short label under each and a highlighted active state; WhatsApp marks the active tab with an unread badge and colour, GRAVID-AMEN with a filled background behind the icon.

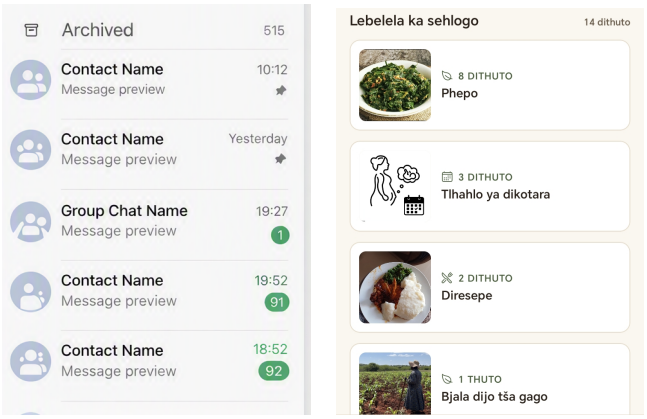


(a) WhatsApp (b) GRAVID-AMEN

Figure 4: The settings list. Both use the same row pattern, a leading icon, a label and a trailing chevron, grouped under headed sections, which is the convention GRAVID-AMEN's settings and profile screens adopted.

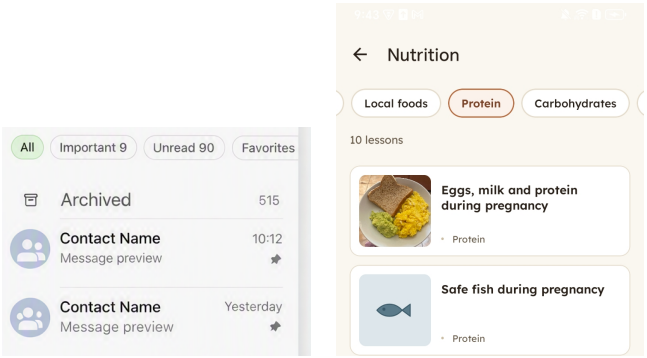
A.6 Design Recommendations Handed to the Receiving Developer

Table 9 is the document given to the receiving developer in the third handover session (Section 7). It was written from the field notes of six facilitators



(a) WhatsApp (b) GRAVID-AMEN

Figure 5: The scrollable content list. GRAVID-AMEN's Food and Guidance tabs present content the same way WhatsApp presents chats, as a single vertically scrolling list of rows, each combining an image, a short label and a count, rather than a grid or a paginated view.



(a) WhatsApp (b) GRAVID-AMEN

Figure 6: The filter chips above a scrollable list, narrowed to the same handful of items in both. GRAVID-AMEN's Food tab reuses the row of pill-shaped chips WhatsApp uses above its Chats list.

across the three clinic days and from the record of the symposium held immediately afterwards. Every change is given with the observation behind it, and the three groups sequence the work by importance.

A.7 Releases and Outcomes Across Milk Matters

A.8 Field Devices and Installation Outcomes

A.9 Measurement Method

Cold start. The median of four runs is 1452.5 ms with a 10 ms spread, measured with `adb shell am force-stop` followed by `am start -W`, repeated five times and discarding the first run because dex optimisation has not settled straight after installation. Against a target of under three seconds the application reaches an interactive activity in about half that on a budget device. One caveat: `am start -W` measures time to activity launch

Table 9: The design recommendations handed over in session three, grouped as they were given. The evidence column is the field observation behind each change, which is what makes the document a handover artefact and not a list of requests: the recipient can sequence the work without reopening the evidence.

Change to make	Evidence from the field notes
Fix first	
Put borders or shadows on content cards and arrows on list rows	Community health workers found the recipe cards and did not tap them; grandmothers had to be told to tap
Show a short tap demonstration the first time the application opens	One woman tapped every icon in turn until she saw what she wanted
Make every tappable target at least 48 dp, the Android minimum and about the size of an adult fingertip	People missed the back arrow because it was too small, and cards did not look tappable
Make the text larger and let it follow the phone's own font size setting	Community health workers said the text was too small; grandmothers at two clinics said it; it came up again at the symposium
Add a medical cross to the emergency icon and keep it red	People looked for emergency numbers inside the food content
Keep the emergency link in settings	One woman found it there after the tab bar failed her
Make every piece of content reachable two taps from home	One woman opened the trimester screen and could not get back out
Make the back arrow in the application larger	The arrow at the top left was too small to hit, and people used the phone's own back button instead
Make the home tab easier to notice, by labelling it or setting it apart from the others	One woman pressed back from an article and expected to land on home
Write recipe amounts in handfuls, cups or spoons and put grams second	Grandmothers spent a long time working out the amounts on screen; the symposium said the same
Then	
Take gardening out of the food section and give it its own place	A pregnant woman asked for this, and a grandmother asked for it separately
Ask one thing per screen during sign-up and keep the main button in the same position	People tapped the Sepedi option and waited, and never saw the continue button
Open the keyboard automatically on the sign-up fields	People did not know a field had to be tapped first, so entering a name was slow
Let people skip sign-up and go straight to the content	Grandmothers handled the content but struggled to register
Add search once the fixes above are done	Grandmothers and a community health worker asked for search and rated it highly; typing was slow throughout
Stretch	
Add a video feed, one video per screen with a swipe up for the next	Grandmothers used it easily and one had never seen it before; most ranked it near the top

Table 10: What reached users across the MM project, 2016 to 2023, after Talbot [23]. The last column records how each row is known, since the two classes see different things: a successor account is an inheriting developer's report of what they found on opening the system, and is the only observation of a release made after the group that wrote it had moved on.

Version	Outcome	How known
2016	Released on Android. Static, and its contents could not be updated by the organisation, so it went out of date	Successor account
2019	Donor chatroom built and working. Not incorporated, the organisation being concerned it could spread misinformation without moderation	Successor account
2020	Staff-facing web application, dynamic content, authentication and cross-platform support added. Never published to any store	Successor account
2021	Usability improvements, an FAQ and a contact page. Between 10 and 50 downloads over eighteen months. Not promoted, the organisation not being satisfied it was ready	Successor account; usage measured
2023	Inherited rather than rebuilt. Approved and released to both stores in September 2023	Author's own deployment

and not time to interactive content, so work done by the JavaScript bundle after the activity appears is not captured.

Package. Native libraries ship for two architectures, arm64-v8a at 19.9 MB and armeabi-v7a at 13.7 MB (Table 12). Content is under a quarter of the package, so the size target was never reachable by removing content. The fix is distribution rather than code.

Bundles. Bundle sizes are estimated from rates in the shipped package. Video costs about 3.8 MB per minute, audio about 0.76 MB per minute and an article with images about 0.25 MB. A four-article bundle is about 1 MB and meets the 2 MB target; a trimester bundle with two three-minute videos, a minute of audio and four articles is about 24 MB and cannot.

Table 11: Devices captured over USB across the three fieldwork days and the follow-up. Serial numbers and participant device names are withheld. Two further dev-kit handsets connected on the preparation day of 19 July and are excluded. One Galaxy A06 had tethering enabled and was probably a researcher's phone, so the participant count in this subset is twenty-three or twenty-four.

Model	Model code	n	Installed
Samsung Galaxy A-series (14)			
Galaxy A01	SM-A015F	1	0
Galaxy A03 Core	SM-A032F	2	2
Galaxy A05	SM-A055F	1	1
Galaxy A06	SM-A065F	3	3
Galaxy A13	SM-A135F	1	1
Galaxy A16	SM-A165F	2	2
Galaxy A17	SM-A175F	1	1
Galaxy A24	SM-A245F	1	1
Galaxy A26	SM-A266B	1	1
Galaxy A33	SM-A336E	1	1
Huawei and Honor (3)			
Huawei nova Y60	WKG-LX9	1	1
Huawei nova Y73S	MGA-LX3	1	1
Honor X5c	NLA-LX2	1	1
Other (7)			
Unbranded Unisoc	—	3	2
OPPO	CPH2591	1	1
OPPO A60	CPH2631	1	1
Redmi A1	220733SG	1	1
Vivo Y03	Y03	1	1
Total		**24**	**22**

Table 12: Package composition. Entry sizes read from the field build using compressed length, which is what each file contributes to the installed size.

Component	Size	Share
Native libraries (40 files, two architectures)	33.7 MB	52%
Compiled Java	9.9 MB	15%
JavaScript bundle	3.5 MB	5%
All content media (video, audio, images)	15.6 MB	24%
Resources, fonts, other	1.8 MB	3%

A.10 Measurement Detail

Table 13: Test device.

Property	Value
Manufacturer	Xiaomi
Model	M2006C3MG (Redmi 9A)
Android version	10
API level	29
Total RAM	2,838,688 kB (2.71 GB)
Free storage	42 GB of 52 GB
Build tested	Pilot build, release configuration

Table 14: Cold start. `adb shell am force-stop` then `am start -W`, repeated five times with a short pause. Run 1 is discarded because dex optimisation has not settled straight after installation. Median of runs 2 to 5 is 1452.5 ms.

Run	TotalTime (ms)	WaitTime (ms)
1 (discarded)	1765	1769
2	1462	1463
3	1453	1456
4	1452	1454
5	1452	1454

A.11 Content Delivery Mechanism

A.12 Test Coverage and Gaps

Each workspace has its own suite, Jest with the `jest-expo/node` preset for the application and Vitest for the dashboard.

Covered. The content repository suite runs against the real bundled content rather than test content. It tests that every language returns only one entry per canonical item, that both languages return the same catalogue size, that an item existing in only one language still appears regardless of the language preference the content is set to, and that a translation shares the bundled media file rather than requiring a second copy. It also tests the content-as-data claim. A tag's value, when set in the dashboard, overrides the category list hardcoded in the source.

Not Covered. The tests are preset to test the web release of the React Native application because there were issues configuring a suitable emulator on the developer's local machine. The native releases of the application are therefore not covered by automated testing and used only manual verification on the test device. There are no component or screen tests in either the application or the CMS. Tests written that spin up an emulator are a definite next step for this system to mature.

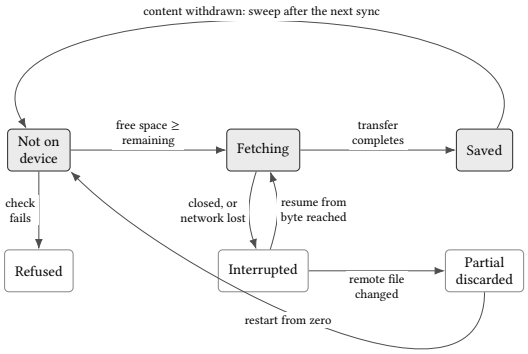


Figure 7: The life of one downloaded file. Shaded states hold bytes on the device and unshaded states hold none. The two paths that matter on a rural connection are the return from Interrupted, which resumes rather than restarts, and the guard on entry, which refuses a download a full phone cannot take.

A.13 Monthly Health Check Email

Figure 8 shows the monthly health check described in Section 6.5 as the project lead receives it.

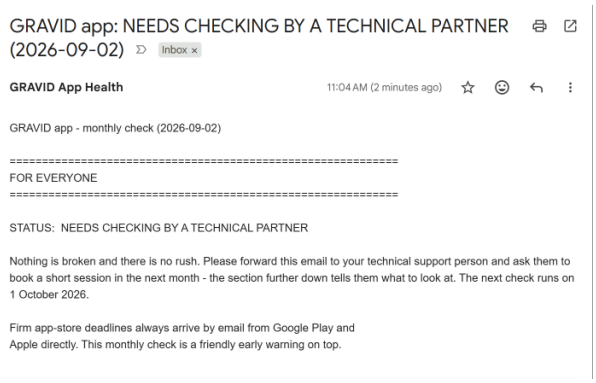


Figure 8: The monthly health check as the project lead receives it. The status is one of three phrases, the technical detail sits below the section addressed to everyone, and the action asked for is to forward the mail.

A.14 Backend Schema

Figure 9 gives the backend schema referenced from Section 4.3.

A.15 On-Device SQLite Schema

Figure 10 gives the local database the application keeps on the device (`gravid.db`, built in `apps/mobile/utils/db.ts`). Two of its five tables, `content` and `tags`, are full-replace mirrors of the Supabase tables in Figure 9 (Appendix A.14), refreshed by `syncContent()` and `syncTags()`; the other three, `download_state`, `queue` and `user_profiles`, are device-local and have no server twin. None of the relationships shown is a SQLite foreign key: because synchronisation re-inserts the mirrored tables wholesale, a real `ON DELETE CASCADE` would take every saved download and queued write with it on each sync, so the references are logical only and orphans are swept after the fact (Appendix A.11).

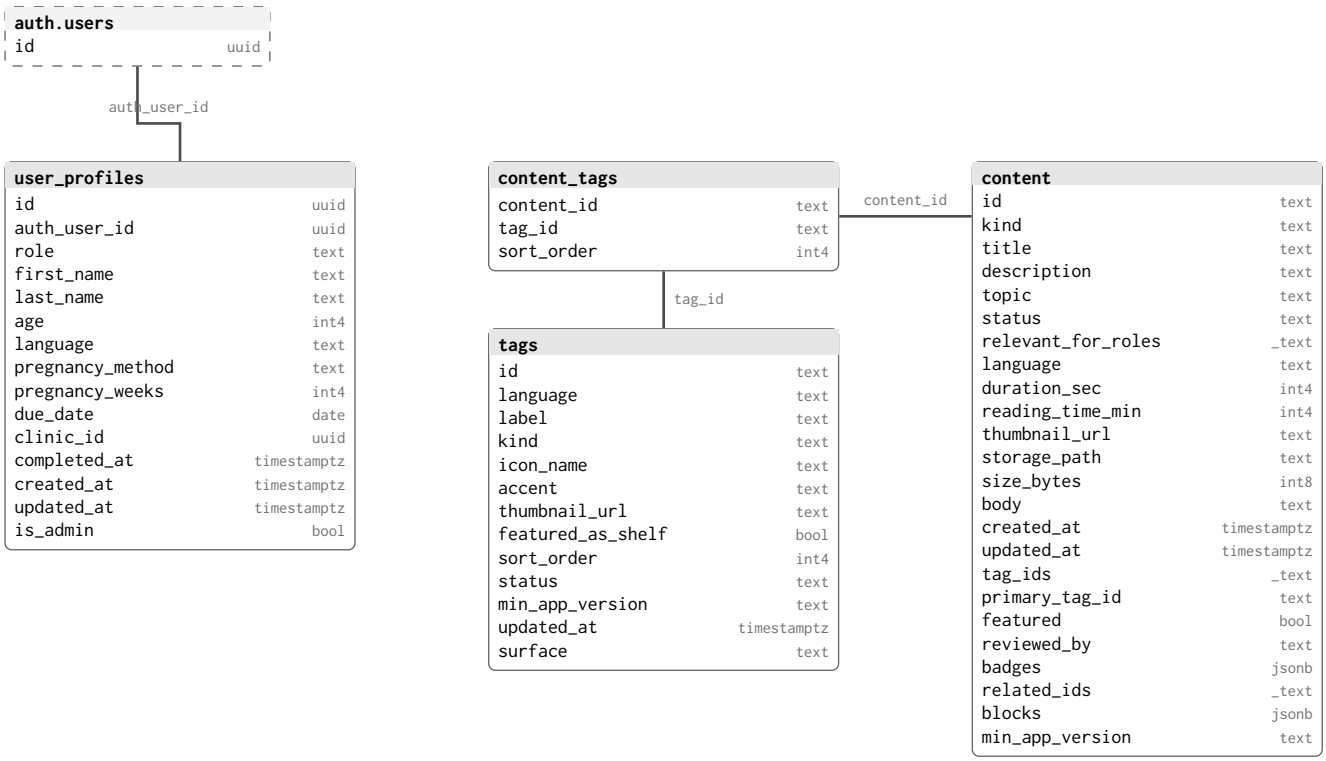


Figure 9: The backend schema. The content and tags tables and the join between them hold the content structure as data, which is what allows a category to be created or moved without a code change.

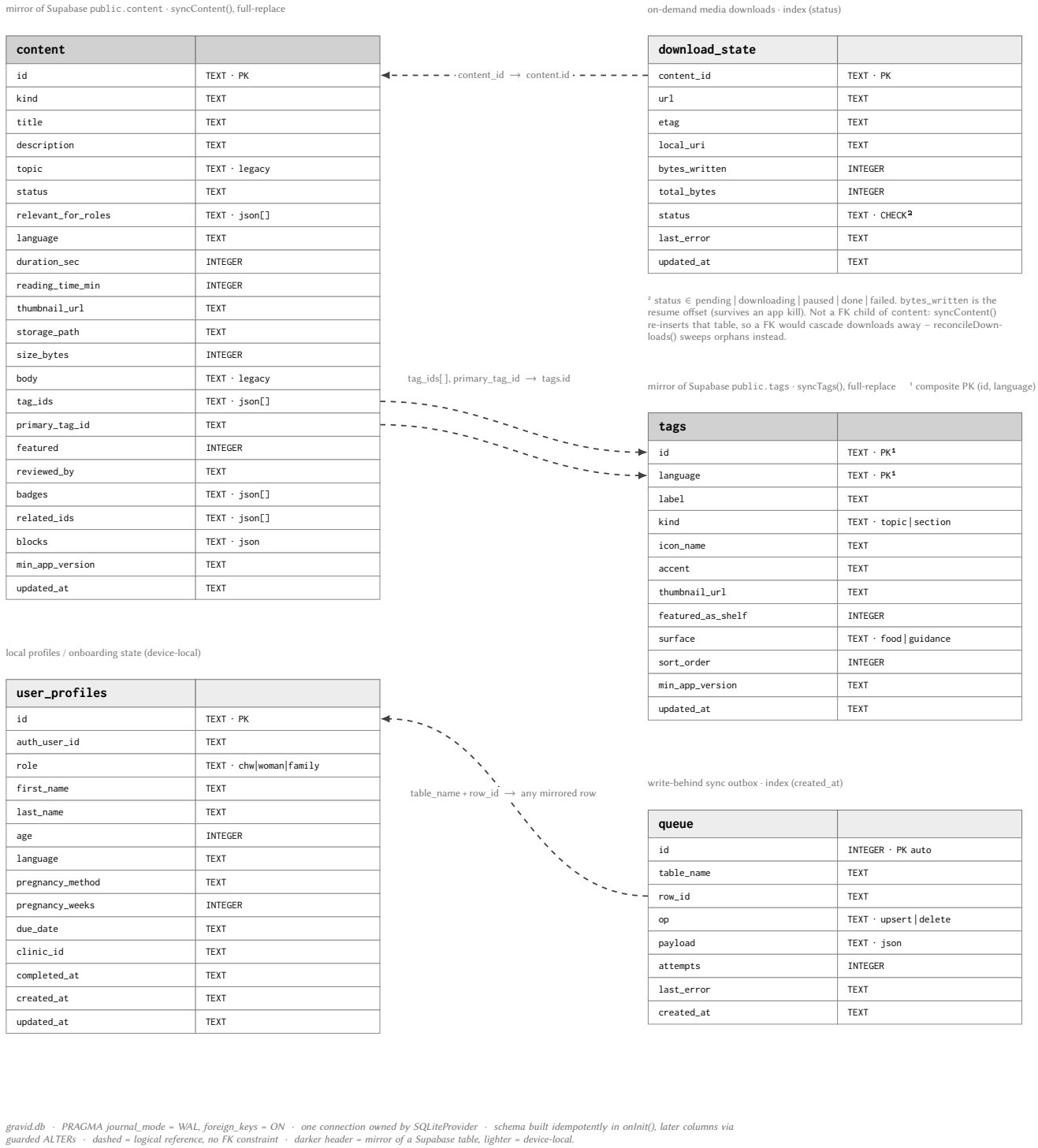


Figure 10: The on-device SQLite schema. A darker header marks a full-replace mirror of a Supabase table (Figure 9); a lighter header marks a device-local table with no server twin. Dashed lines are logical references, not foreign-key constraints: synchronisation rebuilds the mirrored tables on every run, so a cascading key would delete saved downloads and queued writes with them.

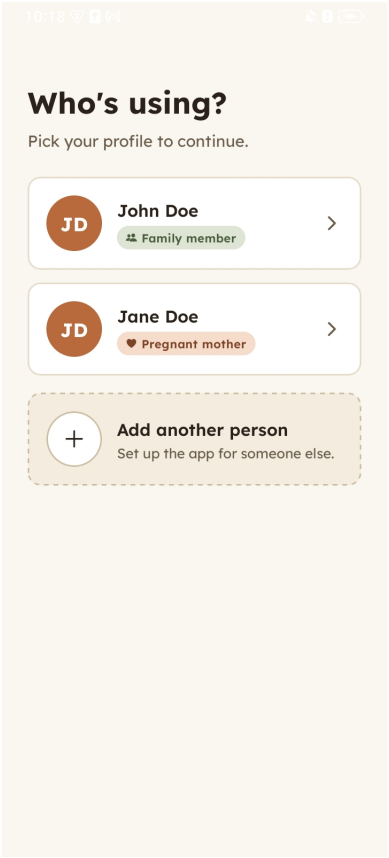
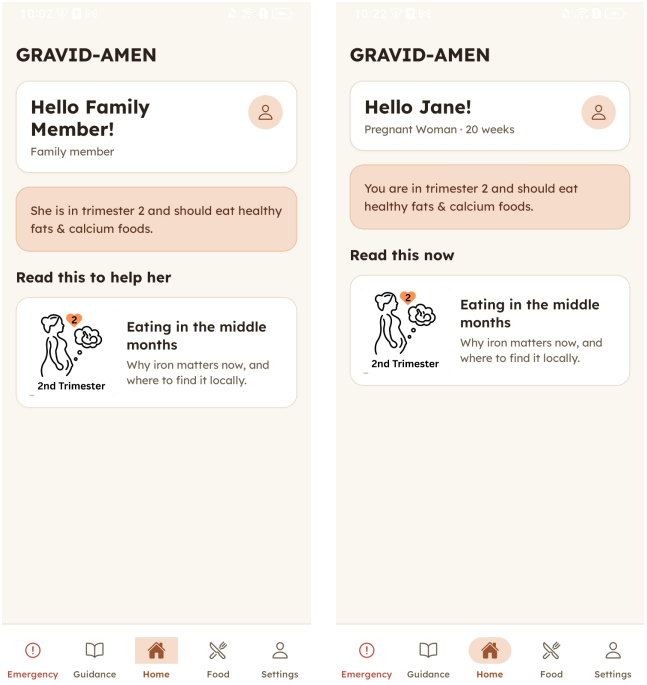
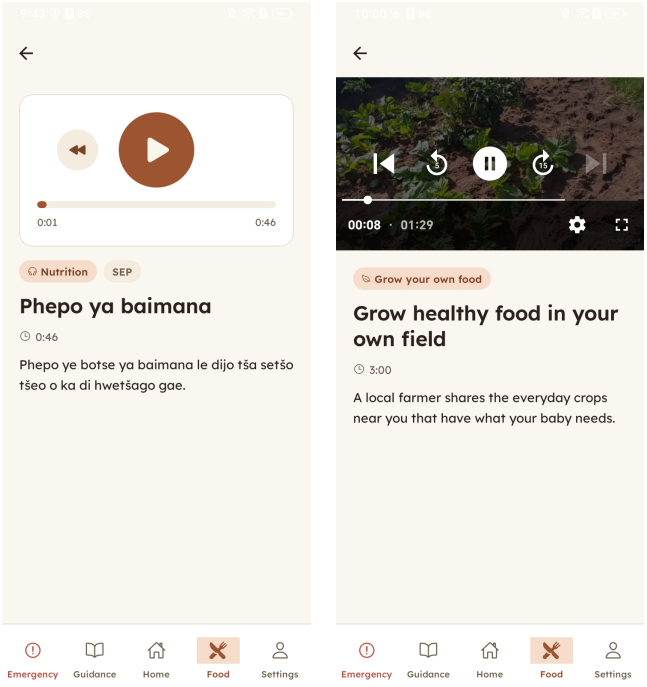


Figure 11: The profile picker read at every launch on a shared device (Section 4.3), modelled on switching profiles on a shared television.



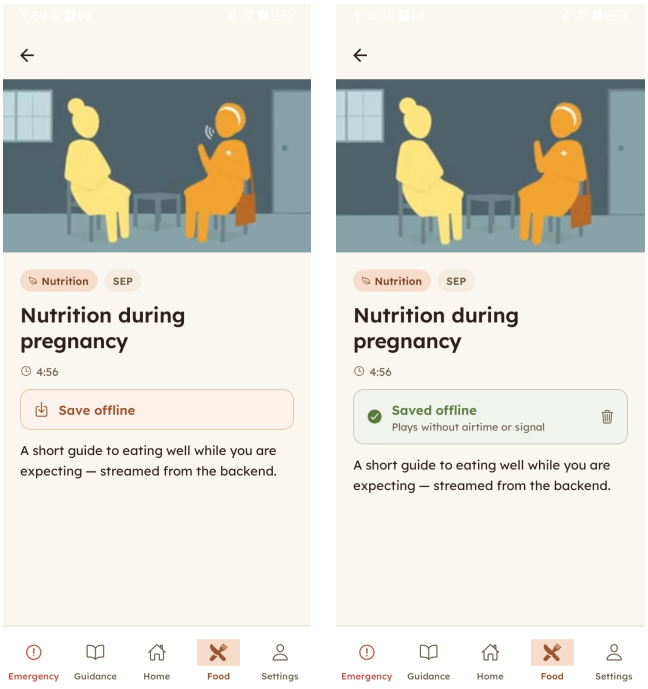
(a) Family member (b) Pregnant mother

Figure 12: The same home screen against two profile roles: a family member reads third-person copy about the pregnancy they are supporting, and the pregnant woman reads the identical fact addressed to her directly.



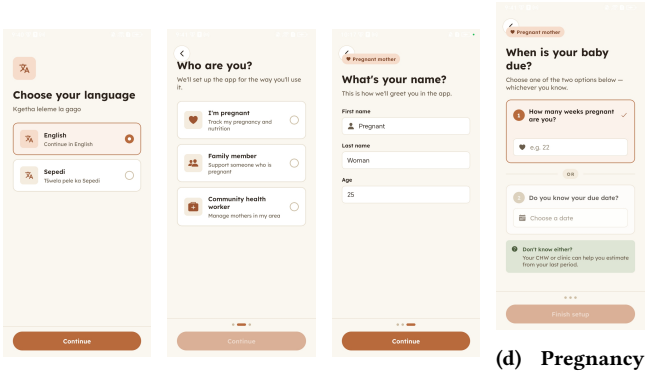
(a) Audio (b) Video

Figure 13: The audio player redesigned for low digital literacy against the video player, which was left with the platform default (Section 4.5).



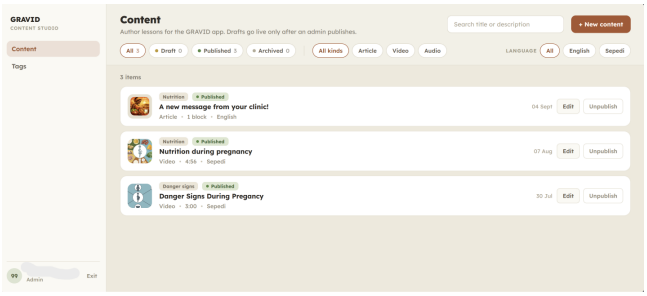
(a) Not on device (b) Saved

Figure 15: One lesson before and after a per-item download, the two states of the mechanism described in Appendix A.11.

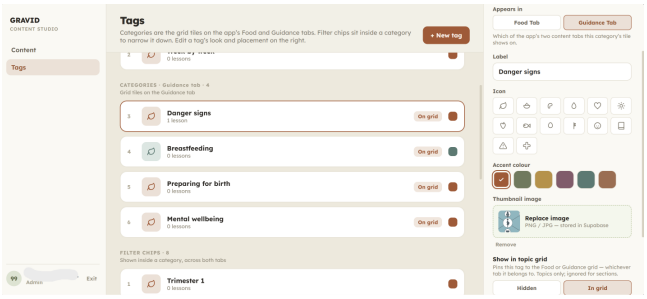


(a) Language (b) Role (c) Name (d) Pregnancy stage

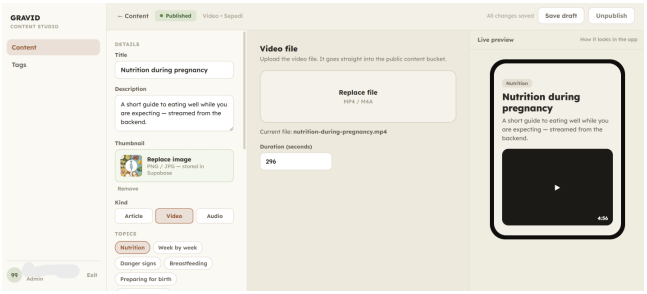
Figure 16: Onboarding for the pregnant-mother role: language, role, name and pregnancy stage. No step asks for a phone number or a password (Section 4.3).



(a) Content list



(b) Tags



(c) Content editor

Figure 17: The content dashboard's content list, tag editor and content editor with its live preview: content and its categories are rows a non-developer edits, not code a developer ships (Section 4.3).

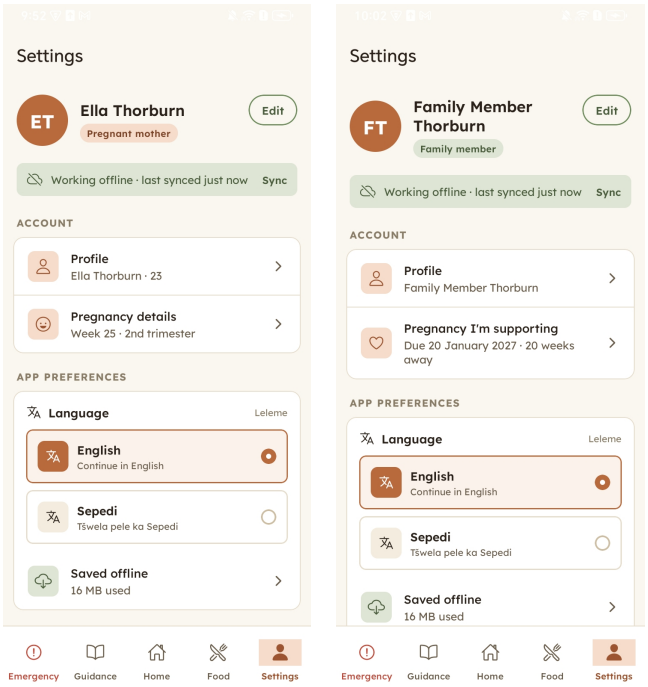
A.16 Application and Content Dashboard Screenshots

A.17 Custody of the Codebase and Environment Variables

The AMEN project lead and the receiving developer both hold administrative access to the database and to the repository, which is the arrangement Talbot and Densmore [24] recommends. The repository holds only the project address and the publishable key, both of which are compiled into the shipped application and the dashboard at build time and are public the moment either is distributed. Access is enforced by row-level security in the database and by an administrator flag in the dashboard rather than by a string that is difficult to share securely. The credentials that confer power, the personal access token, the service role key and the signing secret, are held per person and are never committed, and a new developer is invited to the project via the platform and creates their own.

A.18 AI Usage Statements

Claude Sonnet 5 was used to assist with drawing tables and figures. It was also used to identify places in the original paper where there were repetitions that could be cut. It was used to critique the paper against the provided marking rubrics to suggest improvements. Most of the improvements suggested involved restructuring passages to flow better and adding more evidence for certain claims. These suggestions were taken, text was not provided by the agent to be copied.



(a) Pregnant mother **(b) Family member**

Figure 14: Settings for the two roles that were fully built. Both report the same offline storage figure because downloads follow the device, not the profile reading it.